

Azure Data Factory – Incremental Load Using a Watermark

Step 1: Verify the Resource Group

- Opened resource group DE_1, containing database54456 (SQL server), datafactory54787 (Data Factory V2), sqlserver54564 (SQL database) and storage167565454 (Storage account).

The screenshot shows the Azure portal interface for the resource group 'DE_1'. The left sidebar contains navigation links for Overview, Activity log, Access control (IAM), Tags, Resource visualizer, Events, Settings, Cost Management, Monitoring, Automation, and Help. The main area displays the 'Essentials' section with a 'Resources' tab selected. A table lists the resources within the group:

Name	Type	Location
database54456	SQL server	India South Central
datafactory54787	Data factory (V2)	East US
sqlserver54564 (database54456/sqlserver54564)	SQL database	India South Central
storage16756454	Storage account	East US

At the bottom, it indicates 'Showing 1 - 4 of 4. Display count: auto'.

Step 2: Create the Output Container

- In storage167565454, created the output container, alongside the existing \$logs container, to hold the incrementally-loaded data.

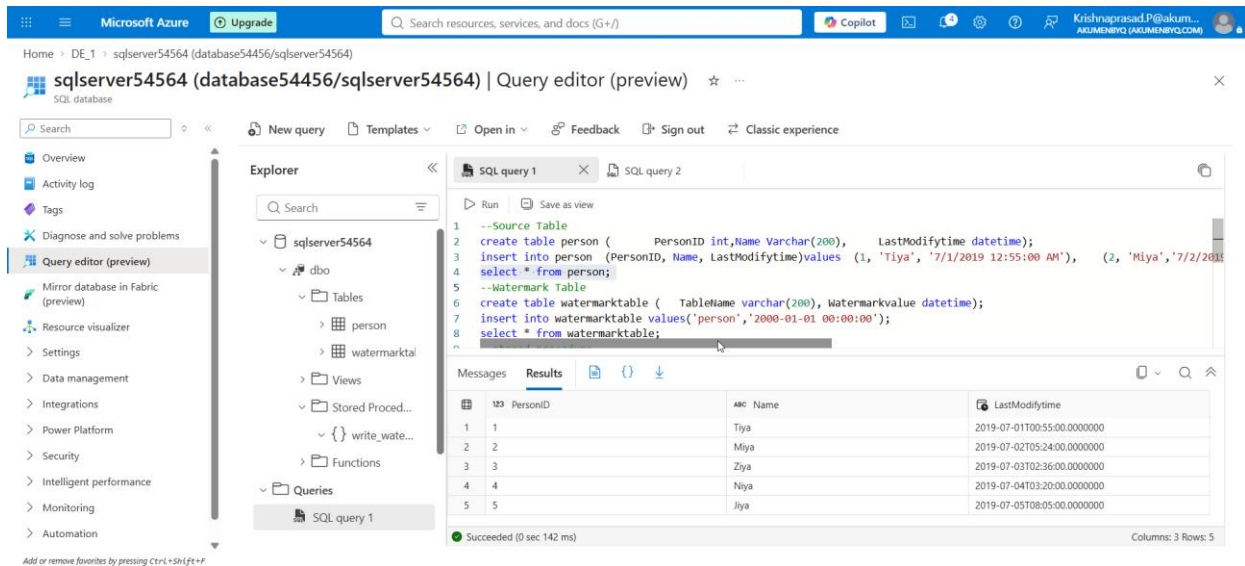
The screenshot shows the 'Containers' page for the storage account 'storage167565454'. The left sidebar lists navigation options including Overview, Activity log, Tags, Diagnose and solve problems, Access Control (IAM), Data migration, Events, Storage browser, Partner solutions, Resource visualizer, and Data storage. The main area shows a list of containers:

Name	Last modified	Anonymous access level	Lease state	Storage connector
\$logs	8/30/2026, 8:07:42 PM	Private	Available	-
output	8/30/2026, 8:09:46 PM	Private	Available	-

The 'output' container has been successfully created alongside the '\$logs' container.

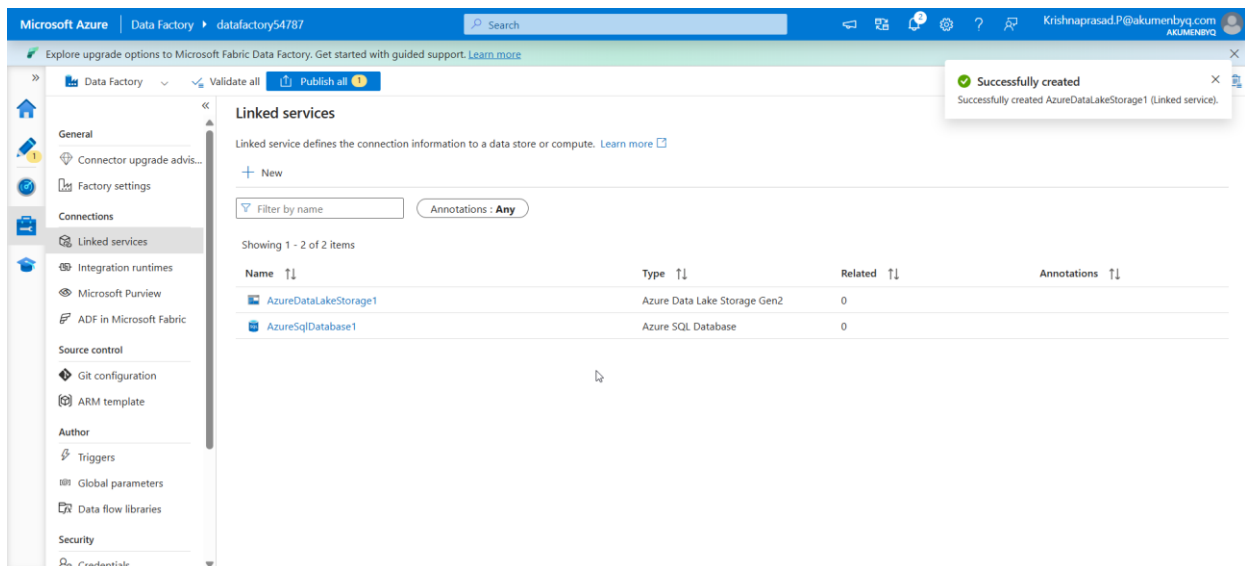
Step 3: Create the Source and Watermark Tables

- Ran a SQL script to create the person table (PersonID, Name, LastModifytime) and inserted 5 rows with staggered LastModifytime values spanning July 1–5, 2019.
- Created a watermarktable (TableName, Watermarkvalue) and seeded it with ('person', '2000-01-01 00:00:00') — an intentionally old date so the first pipeline run captures every existing row.



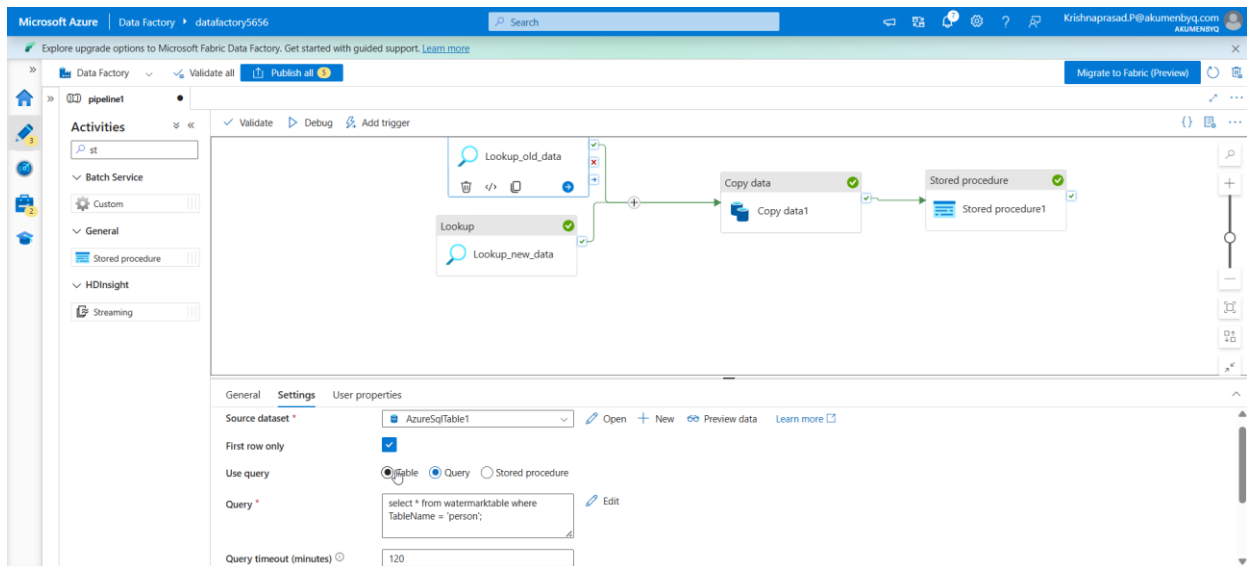
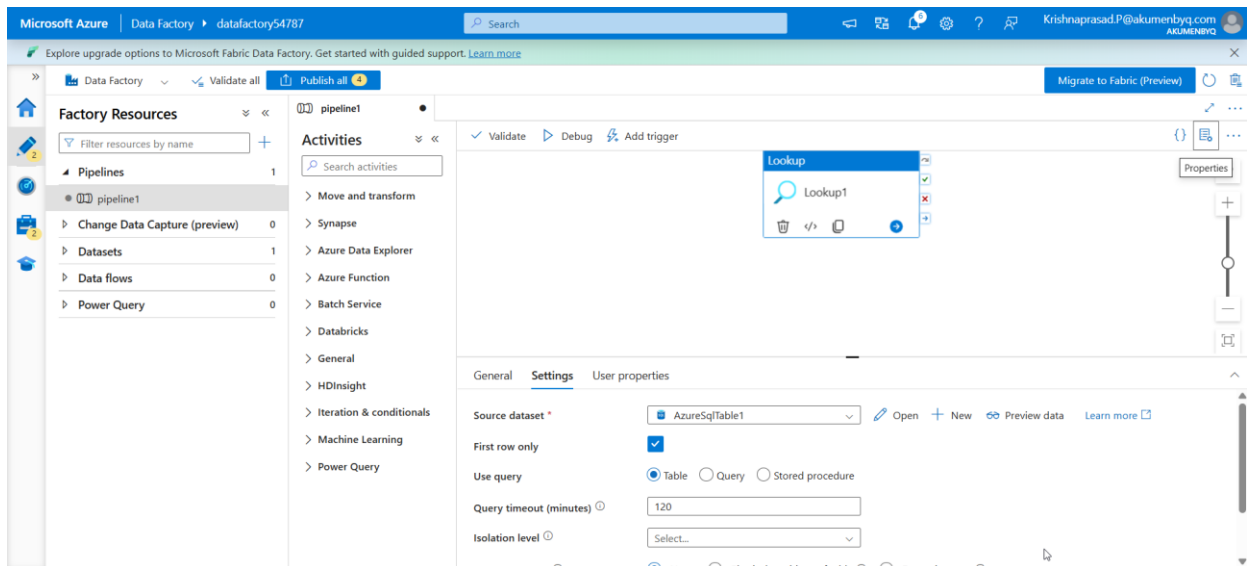
Step 4: Create the Linked Services

- In Data Factory, created two linked services: AzureDataLakeStorage1 (Azure Data Lake Storage Gen2) and AzureSqlDatabase1 (Azure SQL Database).



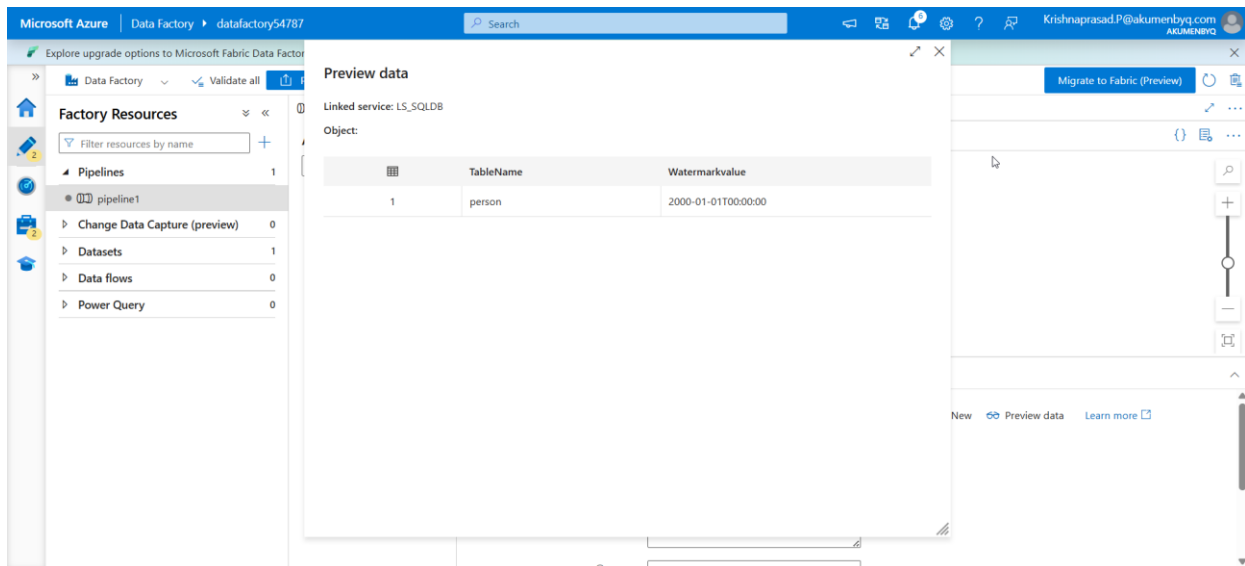
Step 5: Look Up the Old Watermark

- Added a Lookup activity (Lookup_old_data) with First row only checked, running select * from watermarktable where TableName = 'person'; to retrieve the watermark value saved from the last successful run.



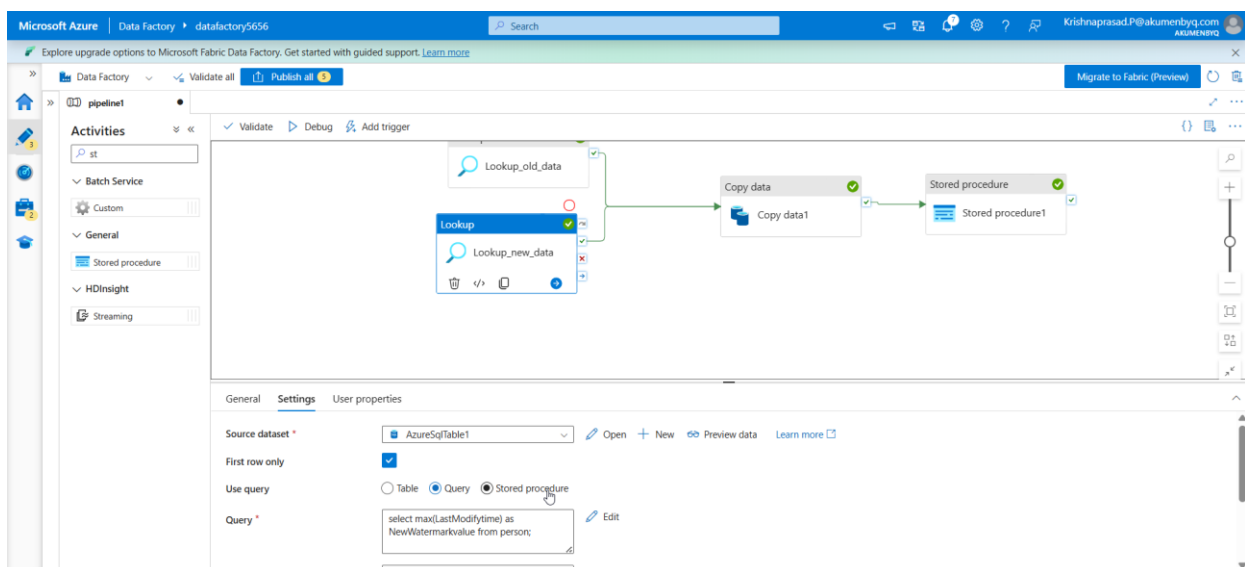
Step 6: Preview the Old Watermark Value

- Previewed Lookup_old_data's output: TableName = person, Watermarkvalue = 2000-01-01T00:00:00 — confirming the seed value is in place before the first run.



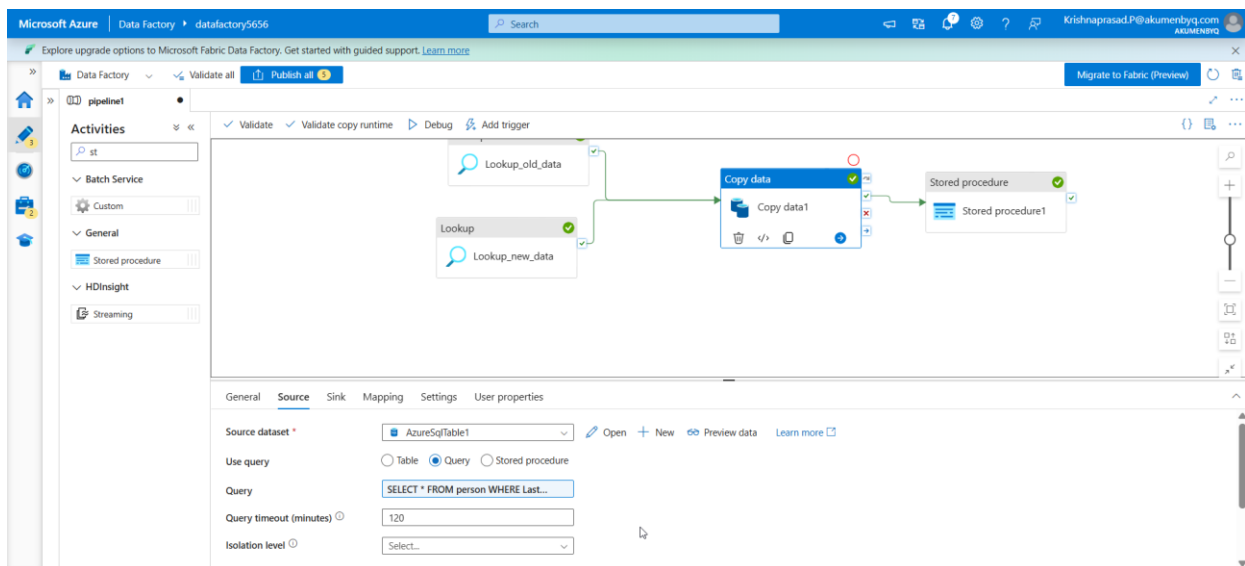
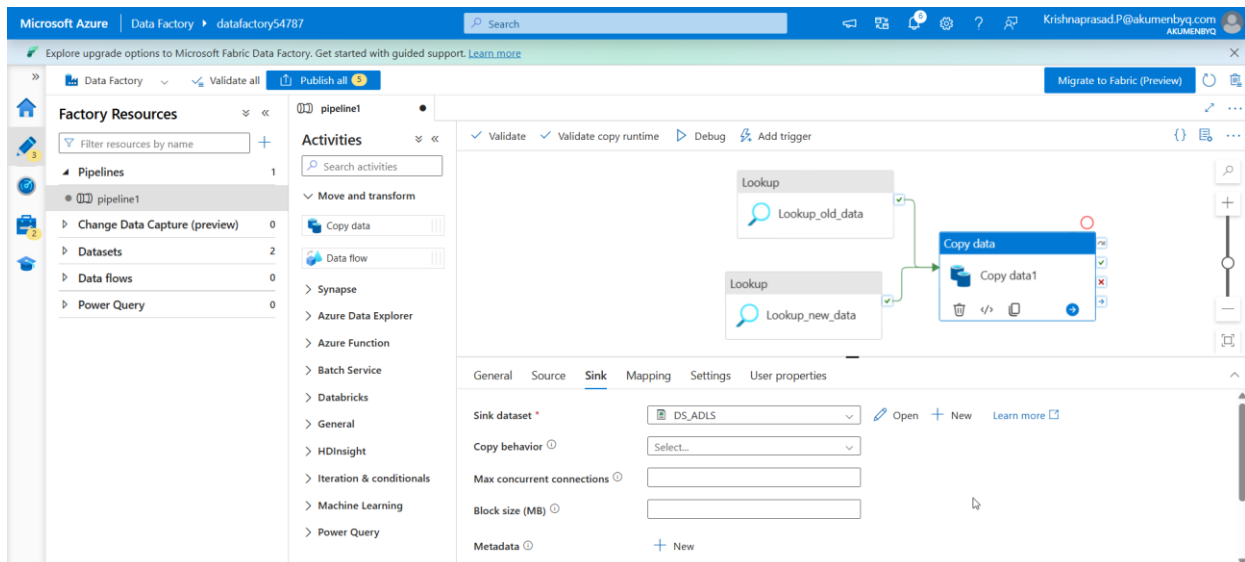
Step 7: Look Up the New Watermark

- Added a second Lookup activity (Lookup_new_data) running `select max(LastModifytime) as NewWatermarkvalue from person`; to find the latest LastModifytime currently in the source table — this becomes the new high-water mark for this run.



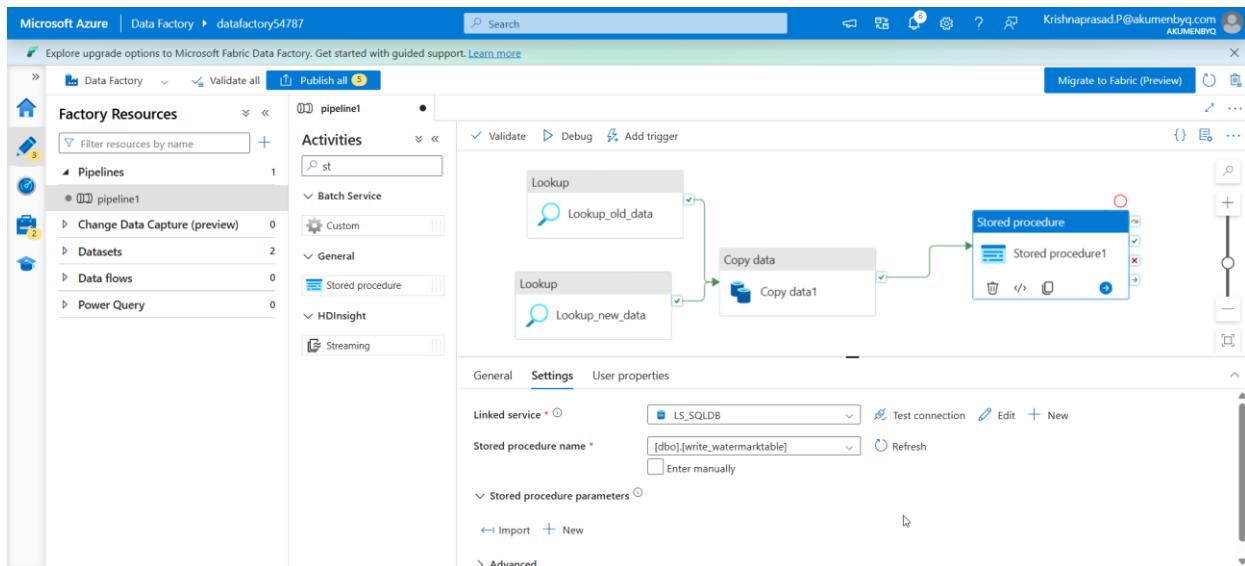
Step 8: Copy Only the Changed Rows

- Configured Copy data1's source query as `SELECT * FROM person WHERE LastModifytime > '@{activity('Lookup_old_data').output...}' AND LastModifytime <= '@{activity('Lookup_new_data').output...}'`, so only rows modified between the old and new watermark are copied, with the sink pointed at dataset DS_ADLS.

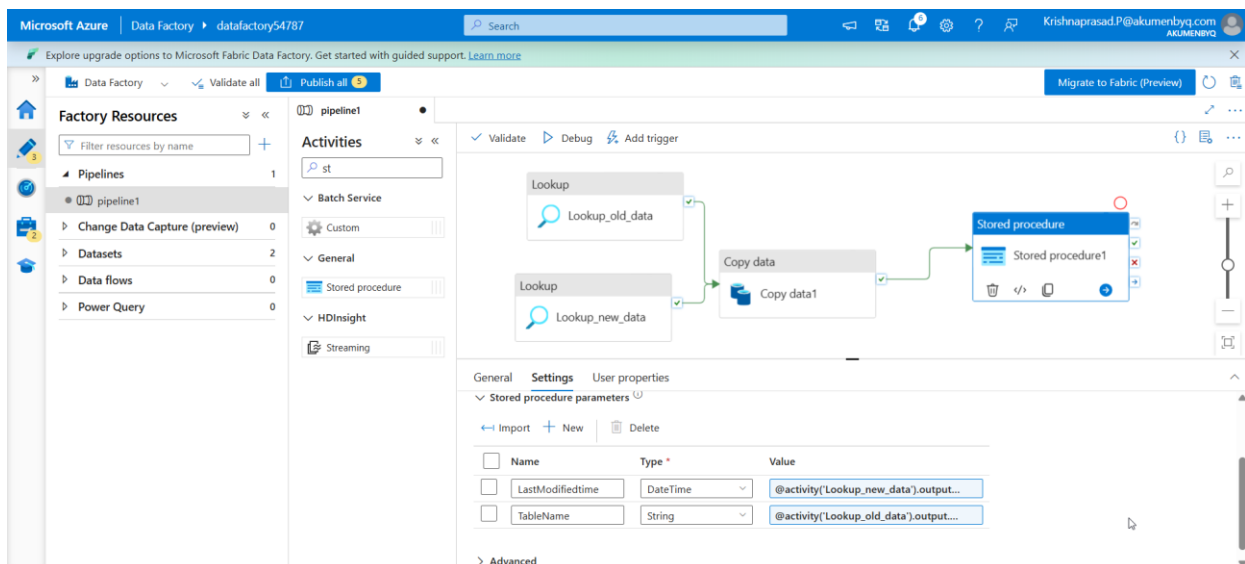


Step 9: Update the Watermark Table

- Added a Stored Procedure activity (Stored procedure1) using linked service LS_SQLDB, calling [dbo].[write_watermarktable].

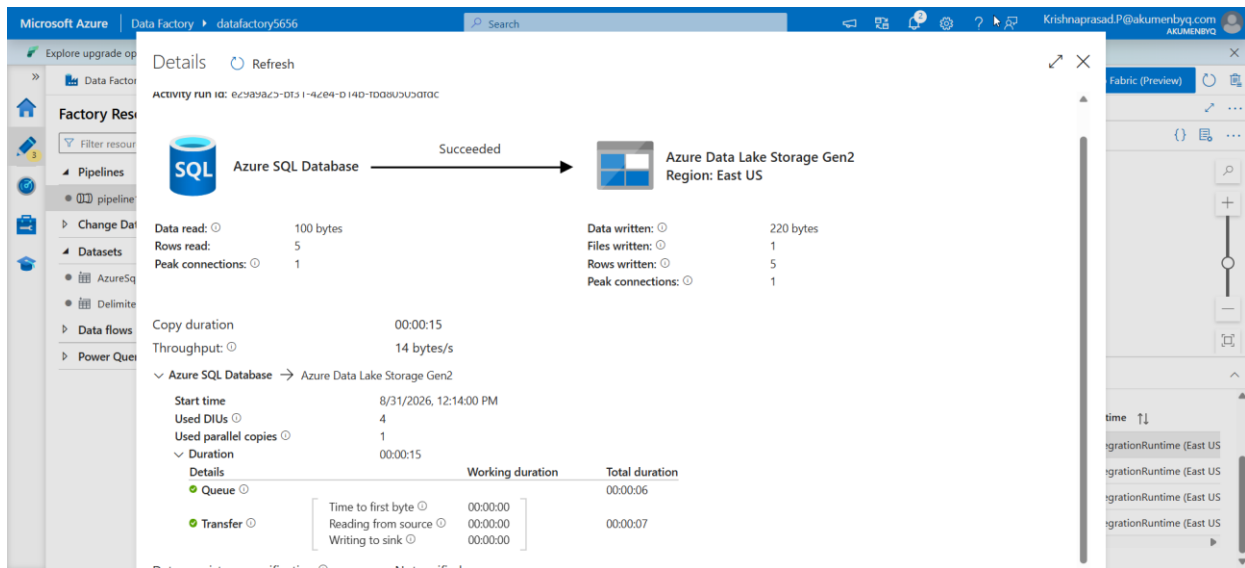


- Passed LastModifiedtime = @activity('Lookup_new_data').output... and TableName = @activity('Lookup_old_data').output... as parameters, so the new watermark is written back for the next run to pick up from.



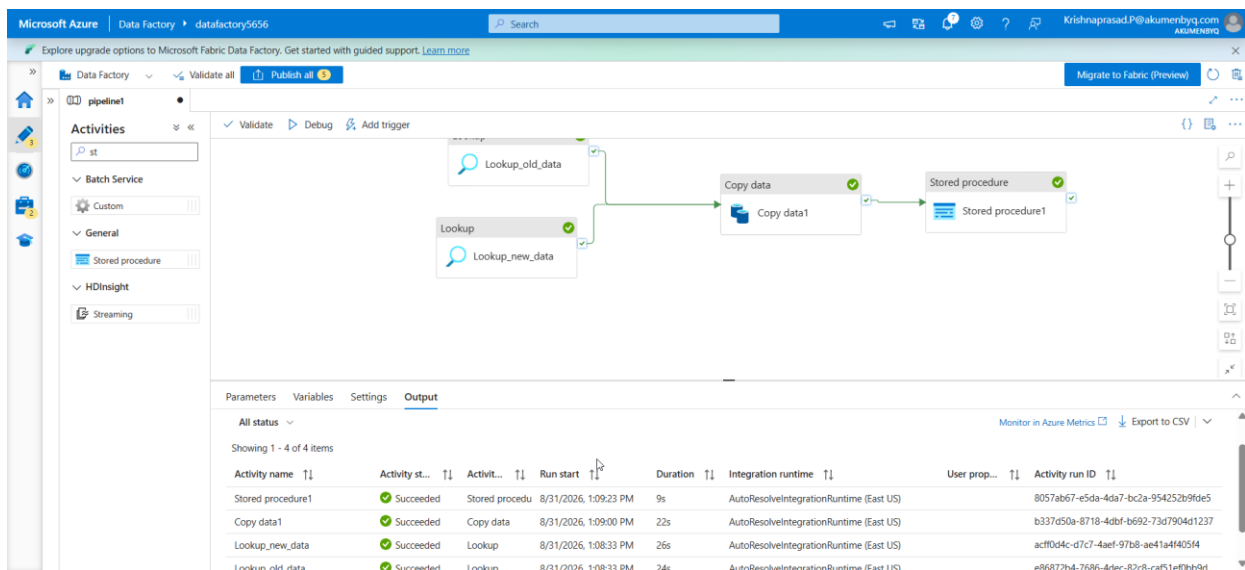
Step 10: Run the Pipeline and Verify the Copy

- Opened the Copy activity's monitoring details for the first run: Azure SQL Database → Azure Data Lake Storage Gen2, Succeeded, 5 rows read and 5 rows written — every row was copied, since the seed watermark was far in the past.

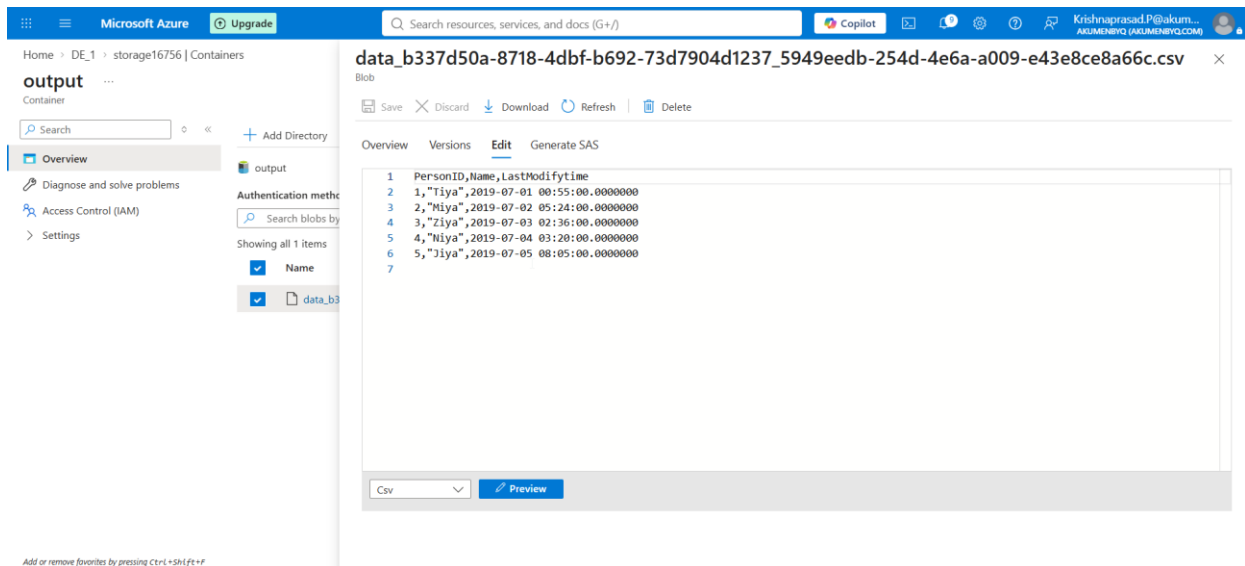


Step 11: Confirm the First Run's Output

- Ran Debug — Stored procedure1, Copy data1, Lookup_new_data and Lookup_old_data all succeeded.



- Checked the output container: the file contains all 5 person rows (Tiya, Miya, Ziya, Niya, Jiya) — the full initial load.



Step 12: Add New Records and Run Again

- After inserting 2 new rows into the person table, ran the pipeline a second time — all four activities succeeded again.

- This time the Copy activity's monitoring details show only 2 rows read and 2 rows written, proving the watermark correctly filtered out the rows already loaded in the first run.

Details Refresh

Learn more on copy performance details from here.

Activity run id: 8ce94c70-e93e-4055-9eae-bf27a1b8179a

Azure SQL Database → **Azure Data Lake Storage Gen2**
Region: East US

Data read: 60 bytes
Rows read: 2
Peak connections: 1

Data written: 116 bytes
Files written: 1
Rows written: 2
Peak connections: 1

Copy duration: 00:00:15
Throughput: 10 bytes/s

▼ **Azure SQL Database** → **Azure Data Lake Storage Gen2**

Start time: 8/31/2026, 12:08 PM
Used DIUs: 4
Used parallel copies: 1

▼ **Duration** 00:00:15

Details	Working duration	Total duration
Queue	00:00:00	00:00:06
Transfer	00:00:00	00:00:07

Data consistency verification: Not verified

Step 13: Confirm Only the New Rows Were Loaded

- The output container now holds two files. Opening the newer one shows it contains only the 2 new rows added after the first run (PersonID 6 and 7, "newrecord") — confirming the incremental watermark pattern works end-to-end.

Microsoft Azure Upgrade

Search resources, services, and docs (G+)

Home > DE_1 > storage16756 | Containers

output ...

Container

Search

+ Add Directory

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

Authentication method

Search blobs by

Showing all 2 items

☐ Name

☒ data_8ce94c70-e93e-4055-9eae-bf27a1b8179a_103c2d5a-c147-4677-ac51-6bad82989a32.csv

☐ data_b3

data_8ce94c70-e93e-4055-9eae-bf27a1b8179a_103c2d5a-c147-4677-ac51-6bad82989a32.csv

Save Discard Download Refresh Delete

Overview Versions Edit Generate SAS

1	PersonID,Name,LastModifyTime
2	6,"newrecord",2019-07-06 09:05:00.0000000
3	7,"newrecord",2019-07-06 10:58:00.0000000
4	

Csv Preview